

Image Processing for Mechatronics (MCTR1010)

**Faculty of Engineering and Materials Science
German University in Cairo**

Milestone 1

TEAM 6

Youssef Mohamed Abodeb 55-1031

Somaya Magdy 55-5027

Bassam Walid 55-0499

Styven Hany 55-10067

Mostafa Shakweer 56-29282

Hassan Yassar 55-26164

Literature Review: Intelligent Lane Keeping and Changing for Scaled Vehicles

Introduction

Autonomous navigation is an important application of image processing in mechatronic systems, where visual information is used to guide the motion of a mobile robot. In this project, a small-scale mobile robot is developed to follow a lane using a camera-based vision system. The main objective is to detect lane markings from real-time camera images and use the extracted information to generate suitable steering and speed commands for the robot.

The system uses image processing techniques to identify the lane region, remove unnecessary background information, extract the lane markings, and determine the robot's position relative to the lane. The detected lane features are then used to control the robot so that it remains aligned with the path. This makes the project a closed-loop system, where the camera acts as the feedback sensor and the robot continuously corrects its motion based on the processed image output.

The final milestone focuses on completing the full hardware and software integration. The system is required to work as a black box, where images or video frames are taken as input, processed using suitable image processing techniques, and converted into control actions for the motors and actuators. The final implementation must also be tested in different scenarios to validate the closed-loop behavior of the robot.

Vision-Based Perception and Image Processing

The proposed system consists of a 3D Printed Car equipped with a camera, a Raspberry Pi, an Arduino, a motor driver, DC motors, and a steering servo motor. The camera is fixed on the car and captures the road or track in front of the vehicle. These images are sent to the Raspberry Pi, which acts as the main processing unit. The Raspberry Pi applies the image processing pipeline to detect the lane markings and extract the required navigation features.

The image processing pipeline starts by selecting a Region of Interest from the captured frame to focus only on the road area. The image is then converted to grayscale and filtered to reduce noise. Thresholding is applied to separate the dark lane markings from the background, followed by morphological operations to remove noise and improve lane continuity. A sliding window technique is then used to detect and track the lane pixels, and a second-degree polynomial is fitted to model the lane shape. From this detected lane, the lane center and lateral error are calculated.

The extracted lane information is used to determine the required steering correction. If the robot is shifted away from the lane center, the system generates a corrective steering command to move the robot back toward the desired path. The Raspberry Pi sends the calculated speed and steering commands to the Arduino through serial communication. The Arduino then controls the DC motor for motion and the servo motor for steering.

The overall system works in a closed loop. First, the camera captures an image of the lane. Then, the Raspberry Pi processes the image and extracts the lane position. Based on this information, a control command is generated and sent to the Arduino. The Arduino actuates the motors, causing the robot to move and change its position. A new image is then captured, and the process is repeated continuously. This feedback loop allows the robot to adjust its motion in real time and remain within the lane.

Methodology

The methodology adopts a multi-stage computer vision and control pipeline designed for real-time execution on a Raspberry Pi. The process is divided into perception, spatial transformation, and actuation logic.

Perception and Feature Extraction

The system processes incoming frames from the /camera/image_raw topic.

- **Preprocessing:** Images are cropped to a specific Region of Interest (ROI) to eliminate environmental noise and improve processing speed. A Gaussian blur is applied to smooth boundaries before inverse binary thresholding isolates the black lane markings.
- **Path Fitting:** Unlike traditional sliding windows, this implementation utilizes **Cubic Hermite Splines**. Lane pixels are extracted row-by-row, and the spline provides a continuous mathematical model ($x = f(y)$) of the lane, which remains robust even during partial occlusion.

Spatial Transformation and Path Planning

- **Homography:** To account for perspective distortion, four-point homography maps pixel coordinates to a ground plane in meters. This allows the system to operate within a metric space where the lane width is defined as **0.30m**.
- **Centerline Reconstruction:** In "Both Lanes" mode, the centerline is the mean of the two splines. In "Single Lane" mode, the system reconstructs the missing path by applying a **0.15m lateral offset** to the detected boundary.
- **Target Smoothing:** A Kalman-style filter tracks the lookahead target, providing a prediction of the target's next position and filtering out high-frequency detection jitter.

Control Actuation

- **Pure Pursuit:** The steering angle is calculated based on the curvature required to reach the lookahead point, using the vehicle's **0.255m wheelbase** as a kinematic constraint.
- **Command Post-Processing:** Steering commands are smoothed via an exponential moving average ($\alpha = 0.9$) to prevent servo oscillations and ensure fluid vehicle movement.

Lane Detection and Control Pipeline

Final ROS 2 Vision-Based Lane Following System

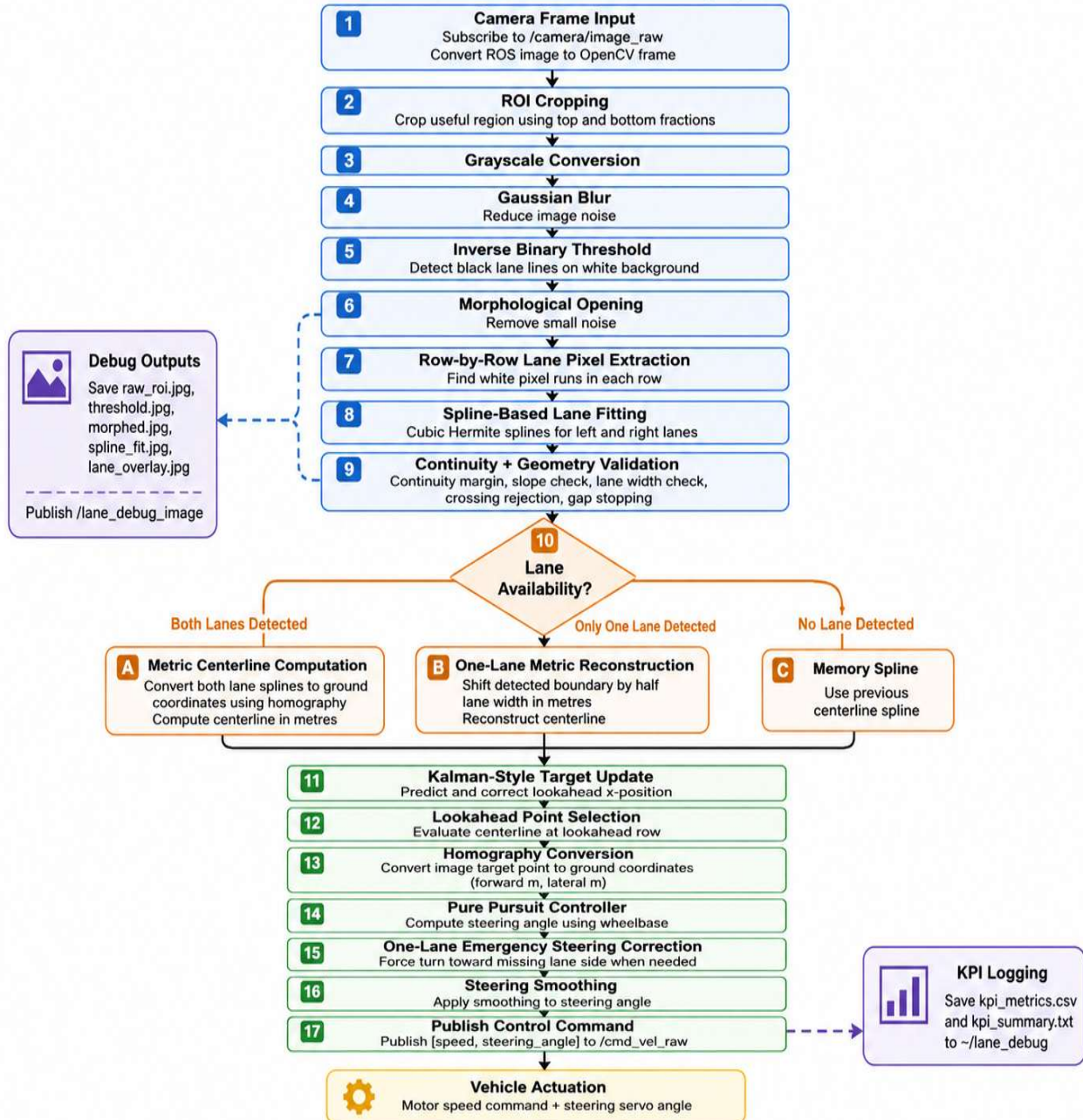


Figure 0: Comprehensive Image Processing and Control Logic Flowchart

Frame_000090

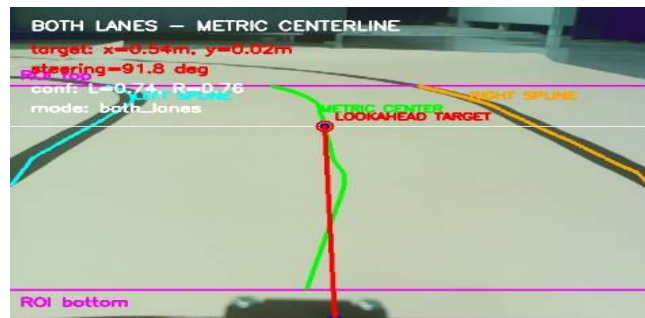


Figure 1 Lane Overlay



Figure 2 Morphed



Figure 3 Raw ROI



Figure 4 Spline Fit



Figure 5 Threshold

Frame_000120

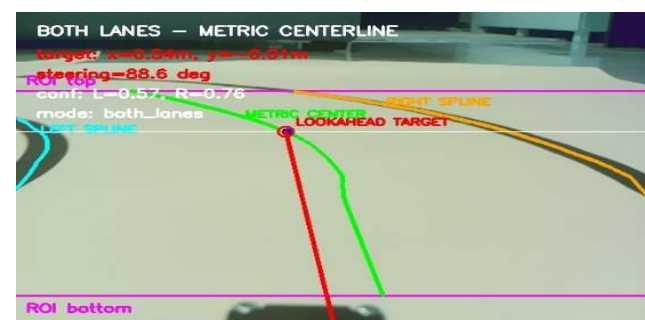


Figure 6 Lane Overlay



Figure 7 Morphed



Figure 8 Raw ROI



Figure 9 Spline Fit



Figure 10 Threshold

Frame_000150

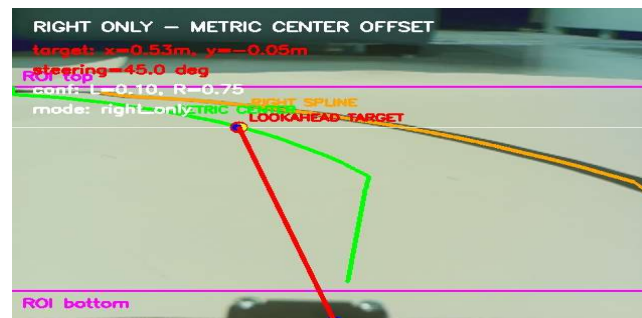


Figure 61 Lane Overlay



Figure 17 Morphed



Figure 18 Raw ROI



Figure 19 Spline Fit



Figure 110 Threshold

Performance Metrics and KPIs

The following KPIs characterize the performance of the image-processing ROS node. These metrics quantify the internal detection and command logic; however, they do not account for physical actuator latency or mechanical slip.

Computational Efficiency (Real-Time Performance)

The system maintained a stable frame rate, ensuring the robot can react to environmental changes within the required temporal window.

- **Average FPS:** 20.82 Hz, providing sufficient frequency for real-time navigation.
- **Mean Processing Latency:** 38.57 ms per frame.
- **Worst-Case Execution Time (WCET):** 57.65 ms, which remains within safe bounds for the defined drive speed.

Perception Robustness and Detection Availability

This section evaluates the reliability of the vision pipeline in various environmental conditions.

- **Target Availability:** 100% of frames resulted in a valid Pure Pursuit target, indicating zero system downtime during the test run.
- **Lane Detection Reliability:**
 - **Dual-Lane Detection:** Successful in 70.02% of frames.
 - **Single-Lane Fallback:** Left-only (7.32%) and Right-only (13.96%) modes successfully reconstructed the path using the 0.30m metric offset.
- **Degraded State Navigation:** The system utilized **Memory Mode** for 8.70% of the run, successfully bridging detection gaps using previous state vectors.

Control Signal Stability

Analysis of the steering and velocity commands issued by the node.

- **Command Continuity:** The average steering change was limited to **1.037 deg/frame**, confirming that the smoothing filters effectively suppressed high-frequency jitter.
- **Steering Range:** The maximum absolute steering angle reached 35.00°, respecting the calibrated kinematic limits of the chassis.
- **Velocity Consistency:** Maintained a constant speed command of 0.700, ensuring predictable momentum during lane transitions.

Project Outcomes and Conclusion

The final system successfully integrates image processing, geometric calibration, lane tracking, pure pursuit control, and hardware actuation into a unified autonomous pipeline. The primary outcomes of this project include:

- **Integrated Lane-Following Pipeline:** Development of a functional autonomous system

capable of detecting black lane markings on a white track and executing steering commands via a Raspberry Pi and ROS 2.

- **Metric-Based Navigation:** Implementation of homography-based conversion, allowing the system to calculate the centerline and pure pursuit target in real-world ground coordinates (meters) rather than raw pixels.
- **Robust Perception Logic:** Use of cubic Hermite splines for smoother lane fitting and a one-lane reconstruction logic that shifts visible boundaries by half the real-world lane width (0.15m) when only one side is detected.
- **Control Stability:** Integration of a Kalman-style filter for target smoothing and a steering smoothing alpha of 0.9 to reduce servo oscillations and prevent sudden jumps in vehicle motion.
- **Reliability Features:** Incorporation of continuity checks, width validation, and missing-gap rejection to prevent lane "jumping" and improve steering stability.

System Limitations

Despite the successful implementation, several technical limitations were identified during the testing and development phases:

- **Environmental Lighting Sensitivity:** The thresholding method relies heavily on the intensity difference between black lanes and the white background; consequently, strong shadows or uneven lighting can degrade detection accuracy.
- **Hardware Calibration Rigidity:** The accuracy of the homography transformation depends on a fixed camera position; any change in camera height or mounting angle requires a complete recalibration of the ground points.
- **Restricted Field of View:** The camera's narrow angle frequently results in frames where only one lane boundary is visible, necessitating heavy reliance on extrapolation and emergency steering correction logic.
- **Computational Constraints:** System performance is bound by the Raspberry Pi's processing power, where increases in image resolution or algorithm complexity can lead to a reduction in the frame rate (FPS).
- **Open-Loop Memory:** While a memory spline is used for short-term stability, it does not currently subscribe to odometry data, meaning the previous centerline cannot be corrected based on actual vehicle displacement.

Conclusion

The project successfully developed a robust, autonomous lane-following system by integrating advanced computer vision techniques with geometric control theory. By transitioning from a basic pixel-based approach to a metric-based architecture, the vehicle achieved a high level of navigation fidelity and reliability.

Key Technical Achievements

- **Metric-Aware Perception:** The implementation of homography allowed the system to ignore camera perspective distortion and operate in a real-world ground plane, using a standard lane width of **0.30m** as a constant reference.
- **Robust Spline Interpolation:** Replacing standard polynomial fitting with **Cubic Hermite Splines** resulted in smoother path planning and more stable steering commands, especially during complex curve transitions.

- **Perception Reliability:** The system demonstrated high resilience by maintaining **100% target availability** throughout the test run. The multi-mode logic—specifically the one-lane reconstruction and memory fallbacks—ensured that the robot remained on track even when the camera's field of view was partially obstructed.

Final Performance Summary

The ROS 2 node met all real-time requirements, maintaining an average frame rate of **20.82 FPS**. This computational efficiency, combined with a **38.57 ms** average processing latency, provided the low-latency response necessary for stable high-speed navigation. The control jitter was successfully minimized, with an average steering change of only **1.037 degrees per frame**, confirming the effectiveness of the Kalman-style target smoothing and exponential moving average filters.

Summary Statement

In summary, the integration of ROS 2, OpenCV, and Pure Pursuit control has produced a system that is not only functional but also adaptable to varying track conditions. While limitations regarding lighting sensitivity and fixed calibration remain, the current architecture provides a solid foundation for future enhancements, such as odometry-based localization and machine-learning-assisted perception.

